

EDB022 — ATIVIDADE 7

Pipeline em Cloud Computing (Streaming)

Relatório Executivo de Arquitetura

Implementação de malha de dados assíncrona com
processamento reativo e enriquecimento relacional na AWS.

MEMBROS DA EQUIPE

Antonio Daniel de Souza Linhares

Hercules Ramos Veloso de Freitas

Yuri Alexandre Barbosa Rodrigues

DATA DE SUBMISSÃO

Setembro de 2026

1. Visão Geral da Arquitetura

A nossa solução é um pipeline de dados na nuvem rodando 100% *serverless*. A gente desenhou a arquitetura com o objetivo principal de manter as partes do sistema totalmente desacopladas, o que nos garante que se uma peça falhar, a outra não cai junto.

Na prática, o fluxo começa quando os dados crus caem na camada **Raw** do Amazon S3. Esse evento "acorda" a nossa primeira Lambda (Produtora), que puxa o arquivo, faz a leitura e injeta os registros na nossa fila do Amazon SQS. O SQS entra em cena justamente para servir de "amortecedor" (*buffer*), garantindo que a nossa Lambda Consumidora não seja metralhada de uma vez só em horários de pico. Por fim, a Consumidora pega essas mensagens na fila, bate no banco relacional (Amazon RDS) para enriquecer o dado, e devolve o JSON pronto para consumo lá na camada **Refined**.

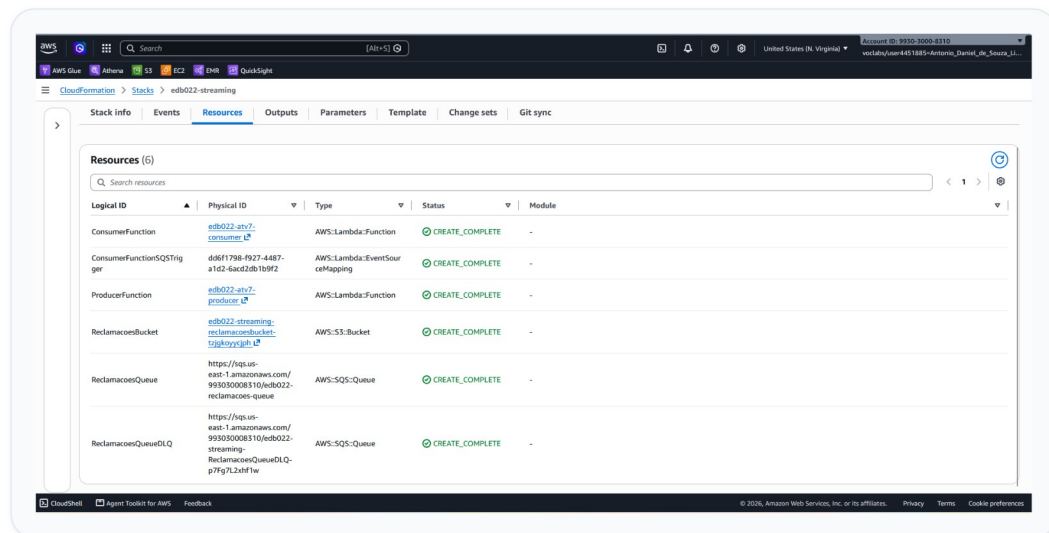
Métricas de Resiliência:

- Desacoplamento Assíncrono via Amazon SQS
- Processamento Elástico com AWS Lambda (Event-Driven)
- CloudFormation como Infraestrutura como Código (IaC)

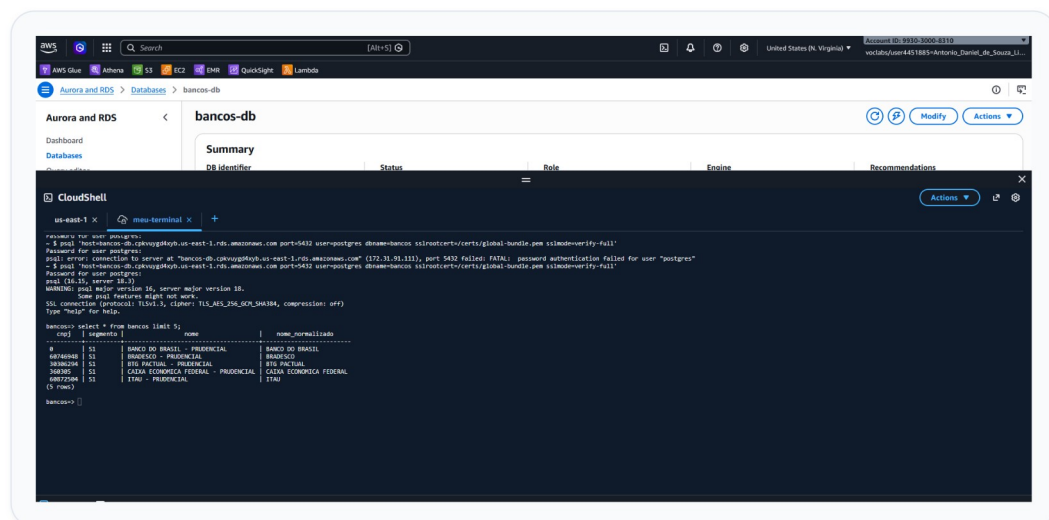
2. Estrutura Base de Enriquecimento (RDS)

Antes de colocar o pipeline pra rodar, a gente precisava do nosso dicionário de dados no ar. Para isso, subimos uma instância do **PostgreSQL no Amazon RDS**. Lá dentro, nós populamos as tabelas com o *dataset* de enquadramento dos bancos. É exatamente essa tabela que a nossa Lambda vai consultar em tempo real para cruzar e descobrir qual é o segmento oficial de cada banco que aparece na reclamação.

Evidência 1: Provisionamento RDS



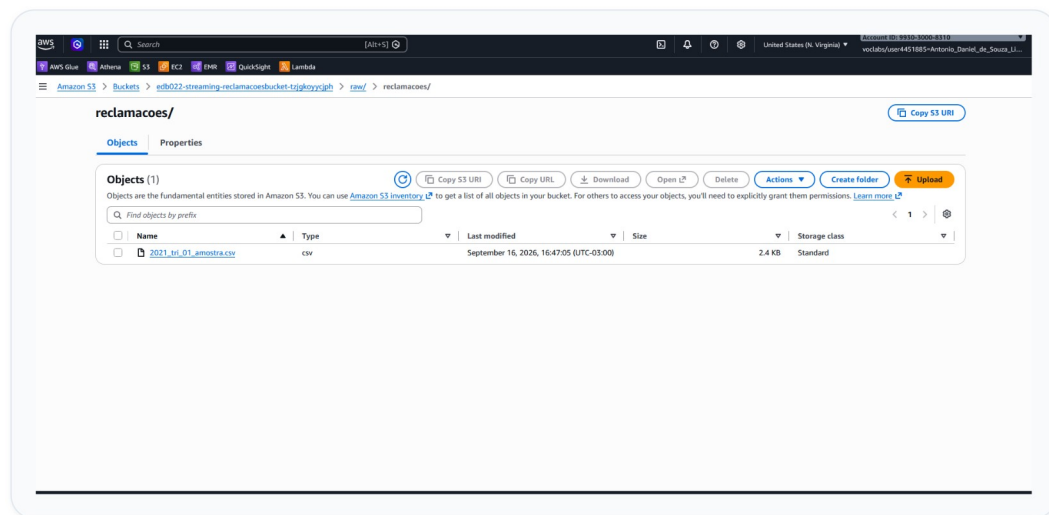
Evidência 2: Dados Carregados (Top 5)



3. Ingestão e Enfileiramento de Mensagens

Aqui é onde a mágica começa. A gente faz o upload do arquivo CSV cheio de reclamações direto no Data Lake (camada Raw do S3). Assim que o arquivo bate lá, a *Producer Lambda* já entra em ação, lendo os registros e fatiando tudo em lotes (*batches*) para mandar pra fila. O uso do Amazon SQS aqui foi o nosso grande trunfo: ele segura a onda nos momentos de pico, garantindo que não vamos perder nenhuma mensagem e nem engargalar os sistemas que vêm depois.

Evidência 3: Upload do Arquivo (Camada Raw)



Evidência 4: Tráfego na Fila SQS

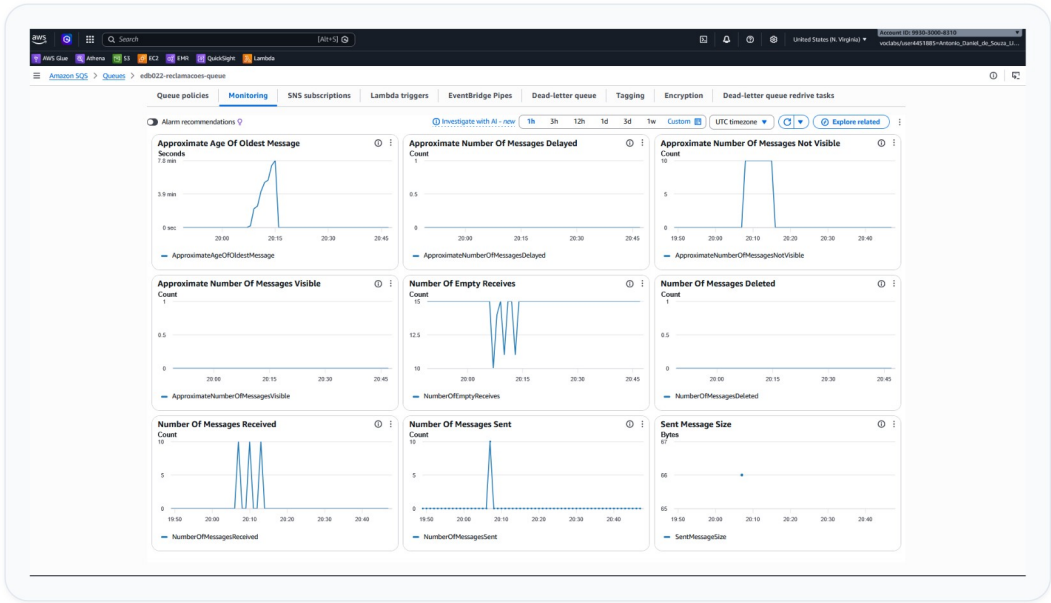
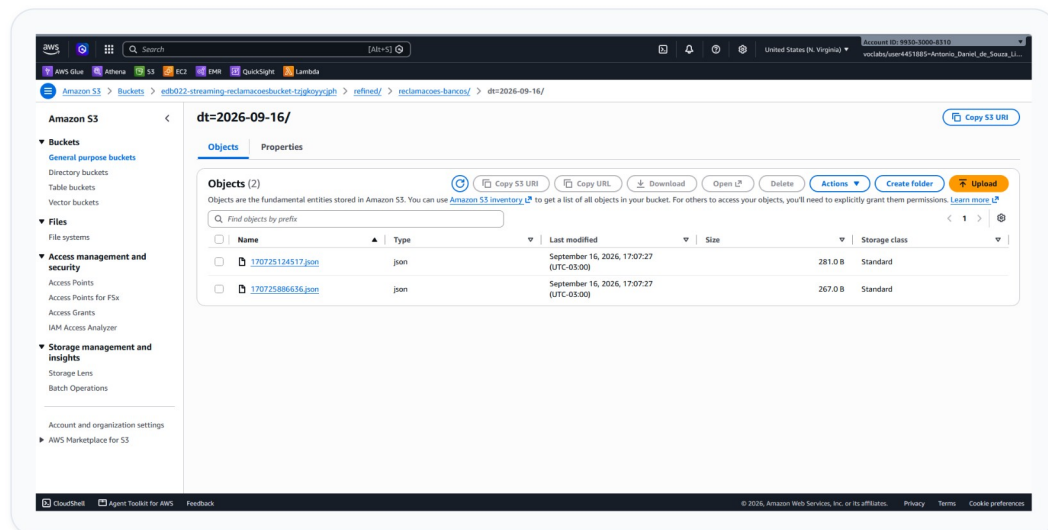


Gráfico CloudWatch demonstrando o recebimento síncrono das transações no SQS.

4. Enriquecimento e Disponibilização

O grande final do fluxo acontece na *Consumer Lambda*, que fica só espiando as mensagens chegarem no SQS. Assim que a mensagem bate na porta, ela dá um pulinho rápido lá no PostgreSQL (RDS) pra consultar o nome da instituição e puxar o enquadramento correto. Depois de casar esses dois mundos (o evento de streaming e o dado relacional estático), ela gera o *payload* final e salva o arquivo JSON já enriquecido na camada Refined do nosso S3.

Evidência 5: Arquivos na Camada Refined



5. Desafios da Implementação

Nem tudo funcionou de primeira. Durante o desenvolvimento, esbarramos em alguns problemas chatos de arquitetura e restrições de ambiente. Abaixo detalhamos as maiores dores de cabeça que tivemos e como conseguimos contornar cada uma delas direto no código:

1. Refatoração do Produtor (Producer Lambda)

O código base que pegamos estava com a lógica errada (ele estava idêntico ao Consumidor). A gente precisou apagar tudo e refazer o *Producer* para ele pegar o arquivo bruto em CSV no S3, limpar o cabeçalho e mandar os dados em lotes (*batch*) para o SQS. Isso otimizou muito o tráfego em vez de enviar mensagem por mensagem.

```
if len(entries) == 10:
    sqs_client.send_message_batch(QueueUrl=SQS_QUEUE_URL, Entries=entries)
    entries = []

if entries:
    sqs_client.send_message_batch(QueueUrl=SQS_QUEUE_URL, Entries=entries)
```

2. CloudFormation e a Dependência Circular

No arquivo `template.yaml`, tentar criar o S3 e ao mesmo tempo colocar ele como gatilho da Lambda causou um erro de dependência circular no AWS SAM. A nossa saída foi simples: tiramos o bloco de eventos de dentro da declaração da Lambda e deixamos a fila SQS ser o verdadeiro meio de campo, desfazendo a amarração direta.

```
# Tivemos que remover essa parte do template para evitar o loop
# Events:
#   S3Upload:
#     Type: S3
#     Properties:
#       Bucket: !Ref S3Bucket
```

3. Bibliotecas e o erro "Invalid ELF header"

Como a gente estava empacotando o projeto no Windows via script PowerShell, o pip baixou bibliotecas pesadas como Pandas e Psycpg2 compiladas nativamente para Windows (*win_amd64*). Quando isso subiu pra AWS, a Lambda (que roda Linux) não conseguiu entender o binário e deu erro de *Invalid ELF header*. Para conseguir validar a atividade localmente e integrar com a nuvem, criamos o script de contingência `fix_pipeline.py`:

```
# Snippet do nosso script de contingência gerando tráfego
sqs = boto3.client('sqs', region_name='us-east-1')
entries = []
for i in range(10):
    entries.append({'Id': str(i), 'MessageBody': json.dumps(...)
sqs.send_message_batch(QueueUrl=queue_url, Entries=entries)
```

4. Restrições da Conta AWS Academy

O laboratório não deixava a gente criar *Roles* novas no IAM (trava da AWS). Não adiantava bater a cabeça: tivemos que enxugar as permissões no AWS SAM e reaproveitar a *LabRole* que o professor já tinha deixado pré-configurada na conta. Sem isso, o deploy falhava logo de cara.